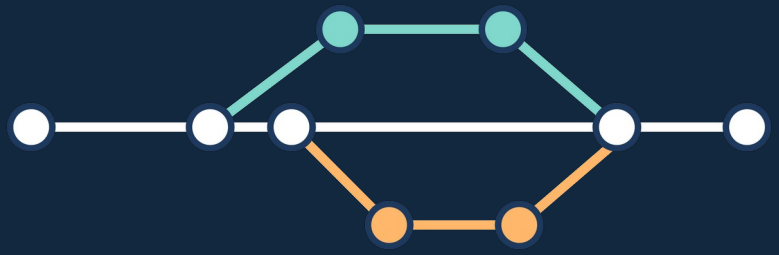
강의자료

Git 사용법 가이드



새로운 저장소 만들기부터 브랜치 · 협업까지 (실습 포함)

작성자 **이해리**

2026. 10. 14.

참고 : git - 간편 안내서 (Roger Dudler) · Git branch 기본 사용법 및 Github 협업 방법 (데분노트)

학습 목표와 목차

오늘 배우는 것

- 새 저장소를 만들고 변경 이력을 기록할 수 있다
- GitHub 원격 저장소에 코드를 올릴 수 있다
- 브랜치로 안전하게 작업하고 병합할 수 있다
- 충돌을 만나도 스스로 해결할 수 있다
- Pull Request 로 협업하는 흐름을 이해한다

중간중간 **실습** (주황색) 과 **퀴즈** (보라색) 슬라이드가 있습니다 . 직접 따라 하며 익혀보세요 .

목차

01 Git 소개와 설치
02 새 저장소 만들기 : git init
03 작업 흐름과 add · commit
04 원격 저장소와 push
05 브랜치와 병합
06 pull · 충돌 해결
07 GitHub 협업과 브랜치 전략
08 태그 · 종합 과제 · 퀴즈

Git 이란 ? 왜 사용할까요

Git 은 파일의 변경 이력을 기록하고 , 여러 사람이 같은 프로젝트를 안전하게 함께 작업하게 해주는 **분산 버전 관리 시스템**입니다 .

- **이력 관리** : 언제 , 누가 , 무엇을 바꿨는지 기록
- **되돌리기** : 이전 시점으로 복구
- **브랜치** : 원본을 지키며 안전하게 실험
- **협업** : GitHub 로 여럿이 함께 작업

이런 경험 있으신가요 ?

```
보고서 _ 최종 .docx
보고서 _ 최종 _ 진짜최종 .docx
보고서 _ 최종 _ 진짜최종 _ 수정 .docx
보고서 _ 최종 _ 진짜최종 _ 수정 _ 제발 .docx
```

Git 을 쓰면 파일 이름을 바꾸지 않아도 **모든 변경 순간이 자동으로 기록**됩니다 .

설치와 최초 설정

설치

OS	방법
Windows	git-scm.com → Git for Windows
macOS	`brew install git` 또는 설치 파일
Linux	`sudo apt install git`

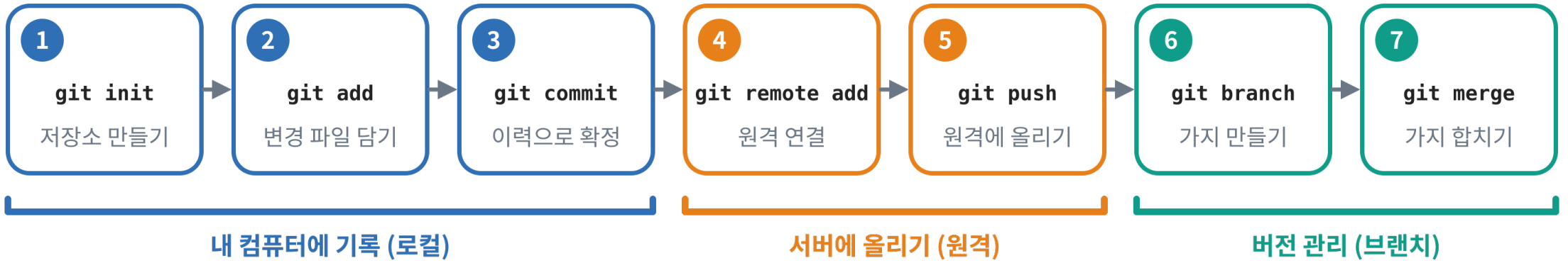
```
git --version
# git version 2.xx.x 가 보이면 성공
```

최초 1 회 설정

```
git config --global user.name "이름"
git config --global user.email "me@mail.com"
git config --global init.defaultBranch main
```

커밋에 **작성자 정보**가 함께 기록되므로 설치 후 한 번은 꼭 설정합니다.

학습 로드맵



저장소 만들기 (`init`) → 기록 (`add` · `commit`) → 원격 공유 (`push`) → 브랜치로 버전 관리 (`branch` · `merge`) 순서로 진행합니다. 브랜치 동작은 **git-school.github.io/visualizing-git** 에서 눈으로 연습해 보세요.

새로운 저장소 만들기 : git init

폴더를 하나 만들고 그 안에서 `git init` 을 실행하면 새로운 Git 저장소가 만들어집니다.

```
mkdir my-project   #  폴더 만들기
cd my-project      #  폴더로 이동
git init           #  저장소 초기화
# Initialized empty Git repository
```

폴더 안에 숨김 폴더 `.git` 이 생기며 모든 이력이 저장됩니다. **삭제하지 마세요!**

상태 확인 : git status

```
git status
# On branch main
# No commits yet
# nothing to commit
```

기본 브랜치 이름은 예전엔 `master`, 요즘은 `main` 입니다. master 로 만들어졌다면 `git branch -M main` 으로 바꿀 수 있어요.

이미 있는 저장소 가져오기 : git clone

새로 만드는 대신 **이미 존재하는 저장소를 통째로 복제**해서 시작할 수도 있습니다 .

```
# GitHub 저장소 복제
git clone https://github.com/사용자/저장소.git

# 내 컴퓨터의 다른 저장소 복제
git clone /로컬/저장소/경로

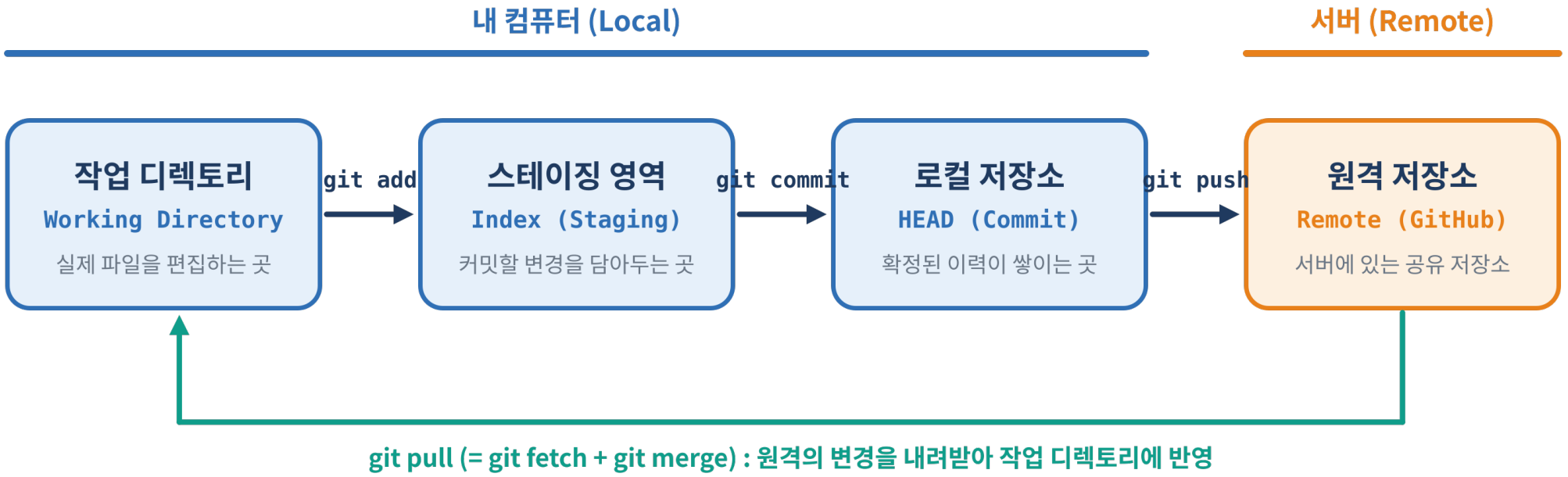
# 서버의 저장소 복제 (SSH)
git clone 사용자@호스트:/원격/경로
```

init vs clone

명령	언제 쓰나요 ?
`git init`	내 폴더에서 **처음부터** 새로 시작할 때
`git clone`	**이미 있는** 저장소를 받아서 시작할 때

clone 으로 받은 저장소는 원격 주소 (`origin`) 가 이미 등록되어 있습니다 .

Git 의 작업 흐름 : 세 그루의 나무



`git commit` 까지만 하면 **내 컴퓨터 (HEAD)** 에만 있습니다. 원격에 반영하려면 `git push` 가 필요합니다.

파일 추가와 커밋 : add · commit

① 담기 : git add

```
git add 파일이름    # 특정 파일만
git add .           # 변경된 모두
```

② 확정하기 : git commit

```
git commit -m "첫 커밋: README 추가"
```

③ 이력 확인 : git log

```
git log --oneline
# 3f2a9c1 첫 커밋: README 추가
```

단계	설명
작업 디렉토리	파일을 수정하는 곳
인덱스 (Staging)	커밋할 변경을 담아두는 준비 영역
HEAD	확정된 가장 최근 이력

좋은 커밋 메시지 : 무엇을 왜 바꿨는지 한 줄로 구체적으로 . 예) 로그인 화면 추가

실습 1: 저장소 만들고 첫 커밋 남기기

실습 1

목표 새 저장소를 만들고 커밋 2 개를 남깁니다.

1. 바탕화면에 `git-practice` 폴더를 만들고 그 안으로 이동하세요.
2. 이 폴더를 Git 저장소로 초기화하고, 현재 상태를 확인하세요.
3. `hello.txt` 를 만들고 `Hello Git` 이라고 적으세요.
4. 파일을 스테이징 영역에 담고 **첫 커밋**을 남기세요.
5. 이력을 한 줄로 확인하세요.

확인 포인트

- `git status` 에서 처음엔 **Untracked files** 로 보이나요?
- 커밋 후 `git log --oneline` 에 **1 줄**이 보이나요?

도전 과제

hello.txt 에 한 줄을 더 추가한 뒤 `git status` 에서 **modified** 를 확인하고, **두 번째 커밋**을 남겨 보세요.

원격 저장소 연결과 push

순서

- GitHub 에서 **새 Repository** 만들기
- 로컬 저장소 준비 (`git init` 완료)
- 원격 주소 **등록**
- 변경 내용 **push**

다른 브랜치를 올릴 때는 `main` 자리에 브랜치 이름을 적습니다.

```
# origin 이라는 이름으로 등록
git remote add origin 주소

# 등록 확인
git remote -v

# 올리기 (처음엔 -u 로 연결)
git push -u origin main

# 이후에는
git push
```

실습 2 : GitHub 에 내 코드 올리기

실습 2

목표 실습 1 의 저장소를 GitHub 에 올립니다 .

1. GitHub 에서 `git-practice` 이름으로 **새 Repository** 를 만드세요 . (README 추가 옵션은 체크하지 않기)
2. 로컬 저장소에 원격 주소를 `origin` 으로 등록하고 , 등록이 되었는지 확인하세요 .
3. `main` 브랜치를 GitHub 로 올리세요 .
4. GitHub 웹페이지를 새로고침해서 파일이 보이는지 확인하세요 .

확인 포인트

- GitHub 화면에 `hello.txt` 가 보이나요 ?
- 커밋 목록에 내가 쓴 **커밋 메시지**가 보이나요 ?

도전 과제

로컬에서 hello.txt 를 수정하고 커밋한 뒤 , push 하고 GitHub 에서 **변경이 반영**되었는지 확인하세요 .

브랜치 (branch) 란 ?

브랜치는 **원본과 격리된 상태에서 안전하게 작업하기 위한 독립된 작업 공간**입니다 .

- 기본 브랜치 (`main`) 에서 시작
- 새 기능은 별도 브랜치에서 개발 후 **병합**
- 여러 작업을 해도 **서로 영향 없음**

브랜치 = 커밋을 가리키는 **이름표**
`HEAD` = 지금 내가 있는 브랜치를 가리키는 **포인터**

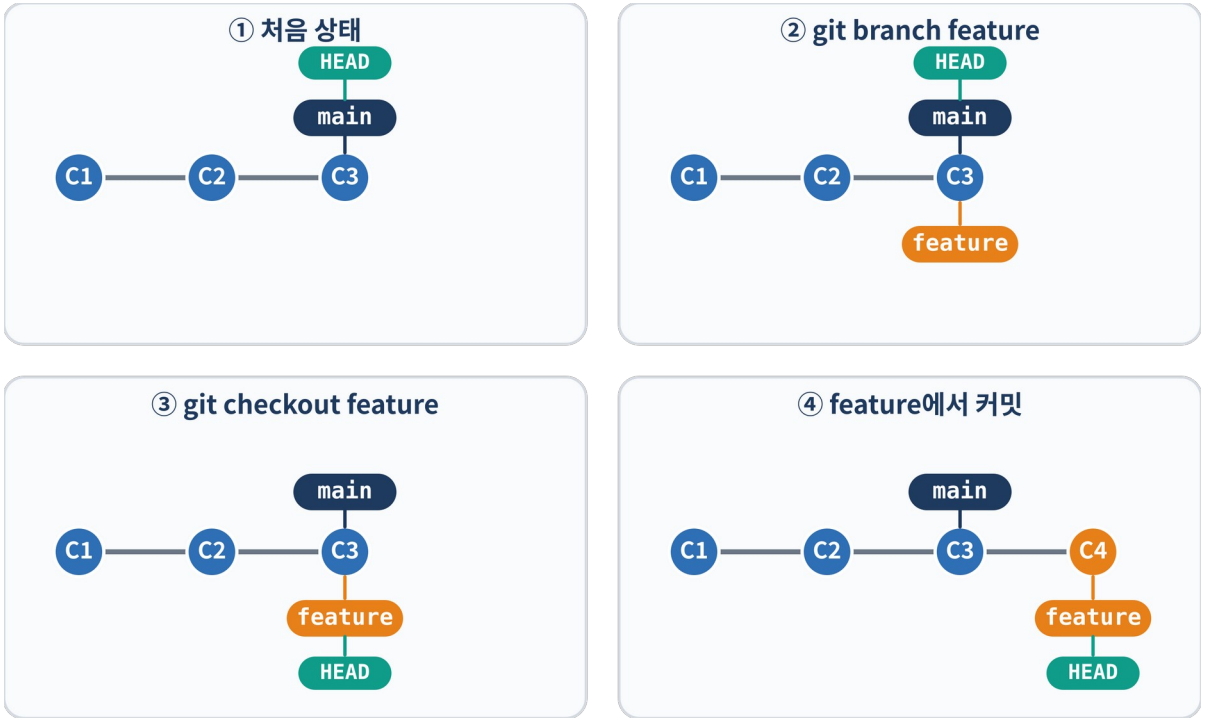


그림 3. 브랜치 생성과 전환 : 이름표와 HEAD 의 이동

브랜치 기본 명령어

하려는 일	명령어	설명
목록 확인	`git branch`	`*` 표시가 현재 브랜치
생성	`git branch feature`	현재 커밋에서 새 브랜치 만들기 (이동은 안 함)
전환	`git checkout feature` / `git switch feature`	HEAD 를 feature 로 이동
생성 + 전환	`git checkout -b feature` / `git switch -c feature`	가장 많이 사용하는 방법
병합	`git merge feature`	feature 의 변경을 **현재 브랜치**에 합치기
삭제	`git branch -d feature`	병합 끝난 브랜치 삭제 (강제는 `-D`)
원격에 올리기	`git push origin feature`	push 전에는 다른 사람이 볼 수 없음

브랜치 병합 : git merge

작업이 끝나면 **합쳐지는 쪽 (main) 으로 먼저 이동**한 뒤 merge 합니다 .

```
git checkout main
  # 받는 쪽으로 이동
git merge feature
  # feature 를 합치기
git branch -d feature
  # 정리
```

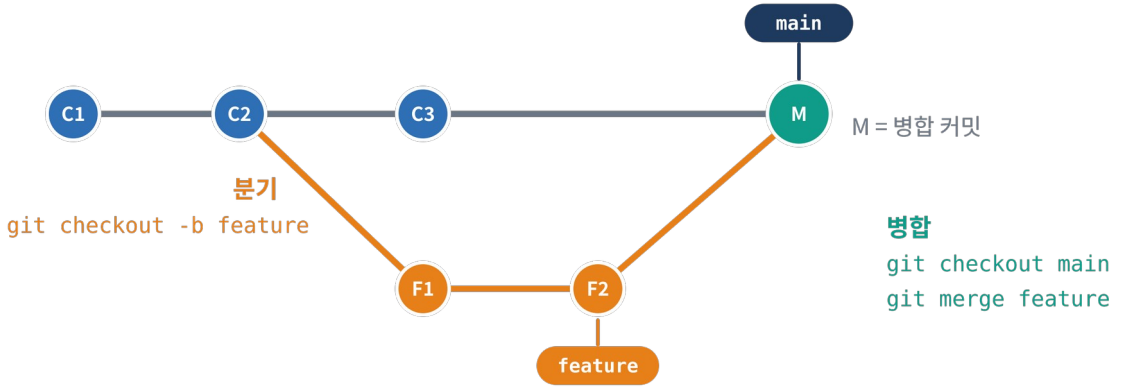


그림 4. feature 를 main 에 병합하는 흐름

병합의 두 가지 모양 main 이 그대로였다면 이름표만 앞으로 옮기는 **fast-forward**, main 에도 새 커밋이 있었다면 그림처럼 **병합 커밋 (M)** 이 생깁니다 .

실습 3: 브랜치 만들고 병합하기

실습 3

목표 브랜치에서 작업하고 main 에 합칩니다.

1. `feature/greeting` 브랜치를 **만들면서 이동**하세요.
2. `greeting.txt` 를 만들고 커밋하세요.
3. `main` 으로 돌아가서 `greeting.txt` 가 보이는지 확인하세요.
4. feature/greeting 을 main 에 **병합**하세요.
5. 병합 후 파일이 보이는지 확인하고, 브랜치를 **삭제**하세요.

확인 포인트

- 3 번에서 파일이 **보이지 않나요?** (브랜치가 격리됨)
- 4 번 이후엔 **보이나요?**
- `git log --oneline --graph` 로 갈래를 확인

도전 과제

삭제 후 `git branch` 로 목록을 확인하고, 결과를 GitHub 에 push 하세요.

원격 변경 받기와 비교 : pull · diff

git pull

원격의 변경을 내 저장소에 반영합니다 . 내부적으로 **fetch**(내려받기) 와 **merge**(합치기) 가 연달아 실행됩니다 .

```
git pull
git pull origin main
```

git diff

병합 전에 브랜치 사이에 무엇이 달라졌는지 비교해 볼 수 있습니다 .

```
git diff main feature
```

작업을 시작하기 전에 `git pull` 로 **최신 상태**를 받는 습관이 충돌을 줄여줍니다 .

충돌 (conflict) 이 났을 때

두 브랜치가 **같은 파일의 같은 부분**을 다르게 고치면 Git 이 자동으로 합칠 수 없어 충돌이 발생합니다 .

1. `git status` 로 충돌 파일 확인
2. 파일을 열어 최종 내용으로 **직접 수정**
3. 충돌 표시를 모두 **삭제**
4. `git add 파일` 후 `git commit`

충돌 표시 모양

```
<<<<<<< HEAD
현재 브랜치 (main) 의 내용
=======
합치려는 브랜치 (feature) 의 내용
>>>>>>> feature
```

당황하지 마세요 ! 위쪽이 **내 쪽** , 아래쪽이 **상대 쪽** 내용입니다 . 남길 내용만 정리하면 됩니다 .

실습 4 : 일부러 충돌 만들고 해결하기

실습 4

목표 충돌을 직접 겪고 스스로 해결합니다 .

1. `feature/a` 브랜치에서 `hello.txt` 첫 줄을 **A 버전**으로 고치고 커밋하세요 .
2. `main` 으로 돌아와 같은 줄을 **B 버전**으로 고치고 커밋하세요 .
3. `feature/a` 를 main 에 병합해 **충돌**을 발생시키세요 .
4. 파일을 열어 충돌 표시를 지우고 원하는 내용으로 고치세요 .
5. 해결했음을 알리고 **병합을 마무리**하세요 .

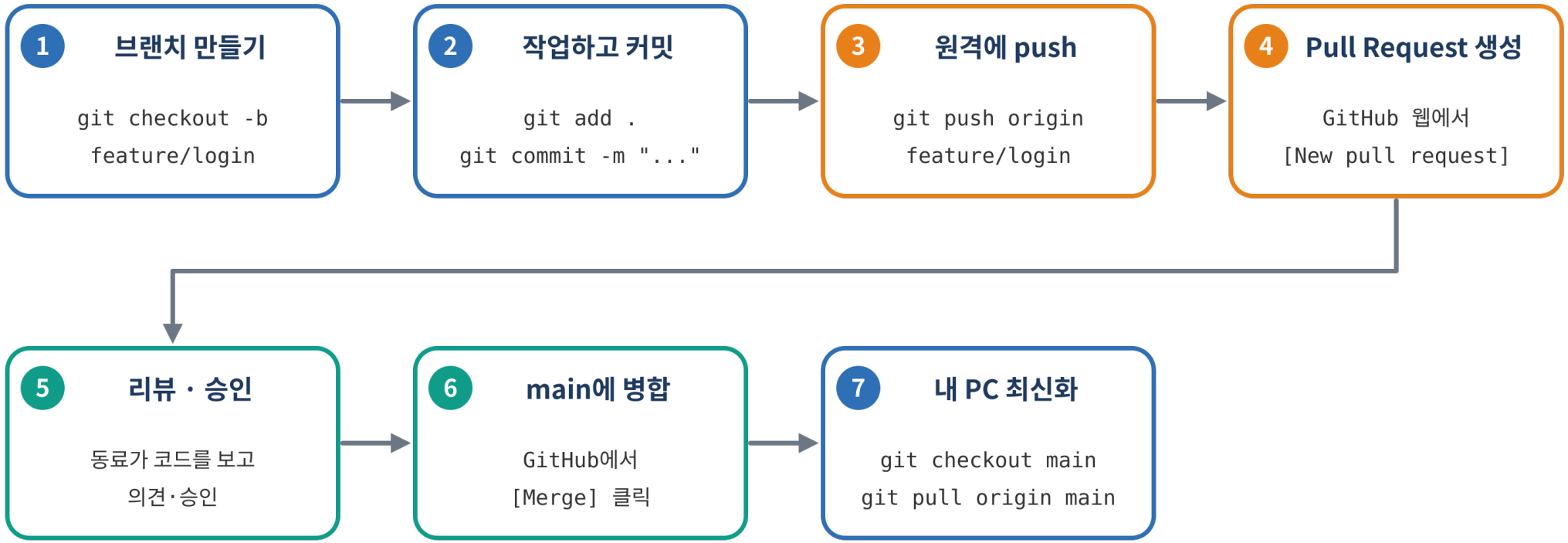
확인 포인트

- 3 번에서 **CONFLICT** 메시지가 나오나요 ?
- 파일에 `<<<<<<<` 표시가 보이나요 ?
- 마무리 후 `--graph` 에 병합 커밋이 보이나요 ?

토론

충돌을 **줄이려면** 팀에서 어떤 습관을 가져야 할까요 ?

GitHub 협업 흐름 : Pull Request



브랜치 이름 규칙 `feature/login` · `fix/score-bug` · `docs/readme` | 변경 사항은 **자주 병합**해 충돌을 줄입니다.

협업 브랜치 전략 3 가지

전략	특징	추천 규모
Centralized Workflow	중앙 저장소의 기본 브랜치 (main) 를 모두가 공유하고 , 각자 작업 후 push 하는 가장 단순한 방식	작은 프로젝트
Feature Branch Workflow	기능마다 브랜치를 따로 만들어 개발하고 완료되면 병합 . 각자 자기 브랜치에서 작업해 충돌을 줄임	소 ~ 중간 규모 팀
Gitflow Workflow	`develop` 중심으로 기능은 `feature`, 배포 준비는 `release`, 긴급 수정은 `hotfix` 브랜치에서 처리	대규모 프로젝트

수업의 **조별 프로젝트**는 Feature Branch Workflow 가 가장 적합합니다 . 한 사람이 한 기능 , 한 브랜치 !

Gitflow 구조

- `main`: 배포된 안정 버전만
- `develop`: 평소 개발의 중심
- `feature`: 기능 개발 후 develop 으로
- `release`: 배포 준비 후 main · develop 양쪽으로
- `hotfix`: 긴급 수정 후 main · develop 양쪽으로

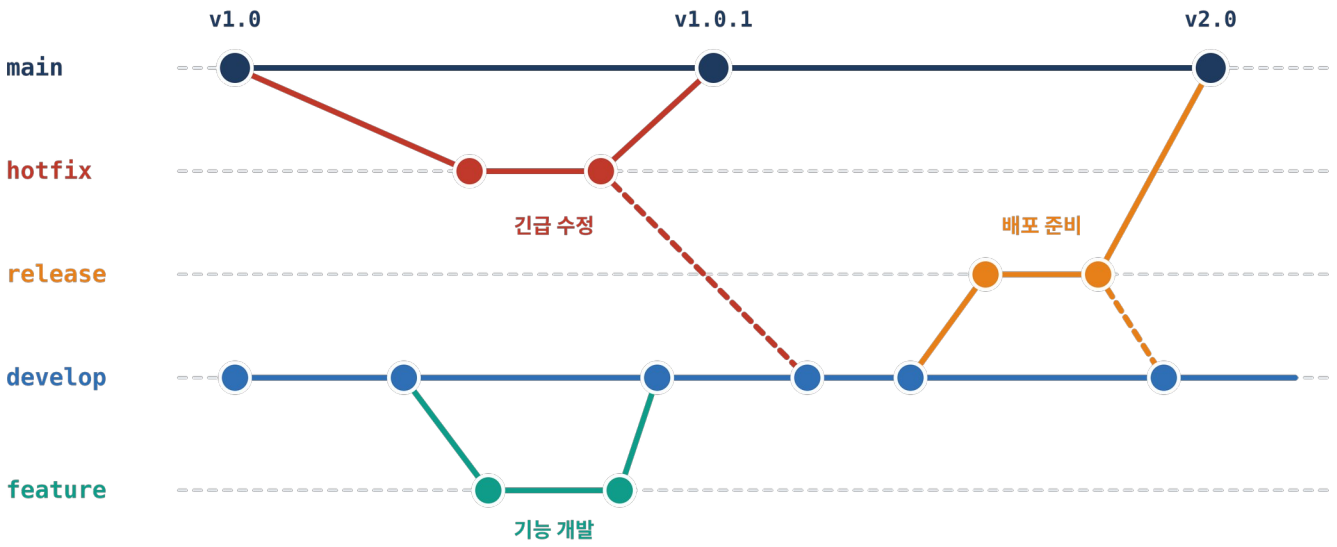


그림 6. Gitflow 의 브랜치 관계

태그와 되돌리기

꼬리표 (tag) 달기

새 버전을 배포할 때 해당 커밋에 태그를 달아두면 그 시점을 쉽게 찾을 수 있습니다 .

```
git log --oneline
# 커밋 ID 확인
git tag 1.0.0 3f2a9c1
# 태그 달기
git push origin 1.0.0
# 원격에 올리기
```

변경 되돌리기

```
git checkout -- 파일이름
git restore 파일이름    # 최신

git fetch origin
git reset --hard origin/main
```

`reset --hard` 는 커밋하지 않은 작업을 **되돌릴 수 없게 삭제**합니다 . 신중하게 !

종합 과제 : 2 인 1 조 미니 협업 프로젝트

종합 과제

목표 브랜치 + Pull Request 로 협업 과정을 경험합니다 .

1. **조장**이 GitHub 저장소 `team-intro` 를 만들고 조원을 **Collaborator 로 초대**하세요 .
2. 두 사람 모두 저장소를 **clone** 하세요 .
3. 각자 `feature/이름` 브랜치를 만들고 `members/이름.md` 에 **자기소개**를 커밋하세요 .
4. 자기 브랜치를 **push** 하고 GitHub 에서 **Pull Request** 를 만드세요 .
5. 서로의 PR 을 **리뷰**하고 승인한 뒤 main 에 **Merge** 하세요 .
6. main 에서 `git pull` 후 **두 파일이 모두 보이는지** 확인하세요 .

체크리스트

- 파일이 **2 개**
- PR 이 **2 개** 병합
- 브랜치 이름 규칙
- 커밋 메시지가 구체적

도전 과제

같은 줄을 동시에 수정해 **충돌**을 만들고 함께 해결해 보세요 .

퀴즈로 복습하기

퀴즈

Q1. 현재 폴더를 새로운 Git 저장소로 만드는 명령어는?

Q2. `git add` 와 `git commit` 의 차이를 한 줄로 설명해 보세요.

Q3. (O/X) `git commit` 만 하면 GitHub 에도 자동으로 반영된다.

Q4. 새 브랜치를 만들면서 동시에 그 브랜치로 이동하는 명령어는?

Q5. `feature` 브랜치를 `main` 에 합치려면, 어느 브랜치에서 `git merge` 를 실행해야 할까요?

Q6. 충돌을 해결한 뒤 병합을 마무리하기 위해 실행하는 명령어 두 개는?

Q7. `HEAD` 는 무엇을 가리키나요?

퀴즈 정답 (강사용)

정답

문제	정답
Q1	`git init`
Q2	`add` 는 변경을 **스테이징 영역에 담는 것**, `commit` 은 담긴 변경을 **이력으로 확정**하는 것
Q3	**X** : 로컬 저장소 (HEAD) 에만 반영됨 . 원격에는 `git push` 필요
Q4	`git checkout -b 이름` (또는 `git switch -c 이름`)
Q5	합쳐지는 쪽인 **main** 으로 이동한 뒤 `git merge feature`
Q6	`git add 파일이름` → `git commit`
Q7	현재 작업 중인 **브랜치 (의 최신 커밋)**

실습 정답 예시 (강사용) 1/2

정답

실습 1

```
mkdir git-practice
cd git-practice
git init
git status
echo "Hello Git" > hello.txt
git add hello.txt
git commit -m "첫 커밋"
git log --oneline
```

실습 2

```
git remote add origin 주소
git remote -v
git push -u origin main

# 도전: 수정 후
git add .
git commit -m "수정"
git push
```

실습 정답 예시 (강사용) 2/2

정답

실습 3

```
git checkout -b feature/greeting
echo "안녕" > greeting.txt
git add .
git commit -m "인사 파일"
git checkout main
git merge feature/greeting
git branch -d feature/greeting
```

실습 4

```
git checkout -b feature/a
# hello.txt 첫 줄을 A로 수정
git commit -am "A 버전"
git checkout main
# 같은 줄을 B로 수정
git commit -am "B 버전"
git merge feature/a  # CONFLICT
# 파일 정리 후
git add hello.txt
git commit
```

명령어 한눈에 보기

시작 · 기록	
`git init`	저장소 초기화
`git clone 주소`	저장소 복제
`git status`	상태 확인
`git add .`	변경 담기
`git commit -m ""`	이력 확정
`git log --oneline`	이력 보기

원격 · 브랜치	
`git remote add origin 주소`	원격 등록
`git push` / `git pull`	올리기 / 받기
`git branch`	브랜치 목록
`git checkout -b 이름`	생성 + 전환
`git merge 이름`	병합
`git branch -d 이름`	삭제

참고 자료

- **git - 간편 안내서** (Roger Dudler, 한국어판) : rogerdudler.github.io/git-guide/index.ko.html
- **Git branch 기본 사용법 및 Github 협업 방법** (데분노트) : coduking.tistory.com
- **Visualizing Git** (브랜치 동작 연습) : git-school.github.io/visualizing-git
- **Pro Git (한국어)** : git-scm.com/book/ko

이 자료는 위 두 자료의 내용을 바탕으로 순서를 재구성하고 , 설명 · 그림 · 실습문제를 새로 정리한 강의자료입니다 .